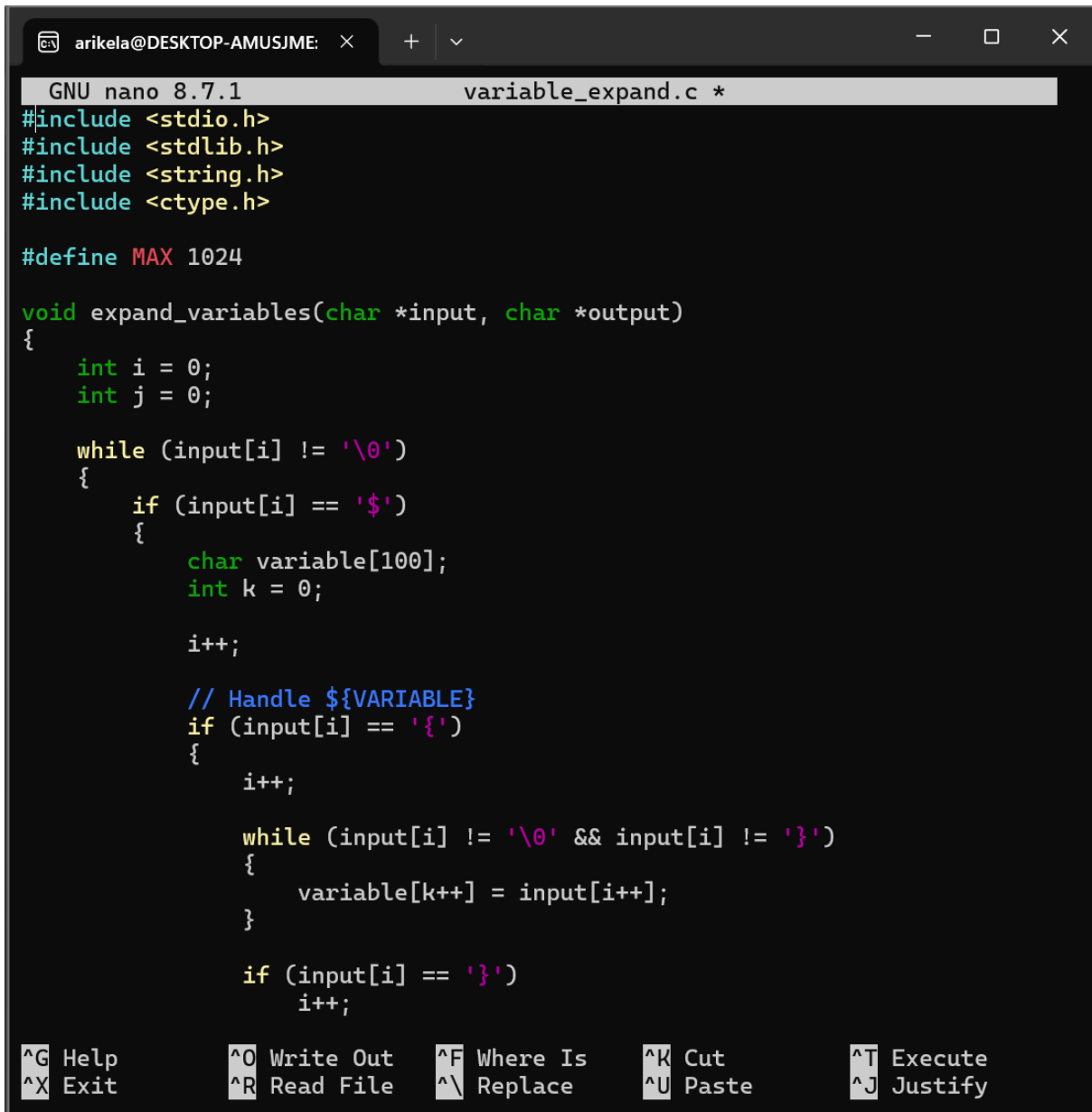


Skill Week 8

Task-1: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic



```
GNU nano 8.7.1 variable_expand.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 1024

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            // Handle ${VARIABLE}
            if (input[i] == '{')
            {
                i++;

                while (input[i] != '\0' && input[i] != '}')
                {
                    variable[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;
            }
        }
        output[j++] = input[i++];
    }
    output[j] = '\0';
}
```

^G Help **^O** Write Out **^F** Where Is **^K** Cut **^T** Execute
^X Exit **^R** Read File **^_** Replace **^U** Paste **^J** Justify

```

        while (input[i] != '\0' && input[i] != '{')
        {
            variable[k++] = input[i++];
        }

        if (input[i] == '{')
            i++;
    }
    else
    {
        // Handle $VARIABLE
        while (isalnum(input[i]) || input[i] == '_')
        {
            variable[k++] = input[i++];
        }
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0; value[x] != '\0'; x++)
        {
            output[j++] = value[x];
        }
    }
    else
    {

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```

        else
        {
            printf("Undefined variable: %s\n", variable);
        }
    }
    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';
}

int main()
{
    char input[MAX];
    char output[MAX];

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("Expanded result: %s\n", output);

    return 0;
}

```

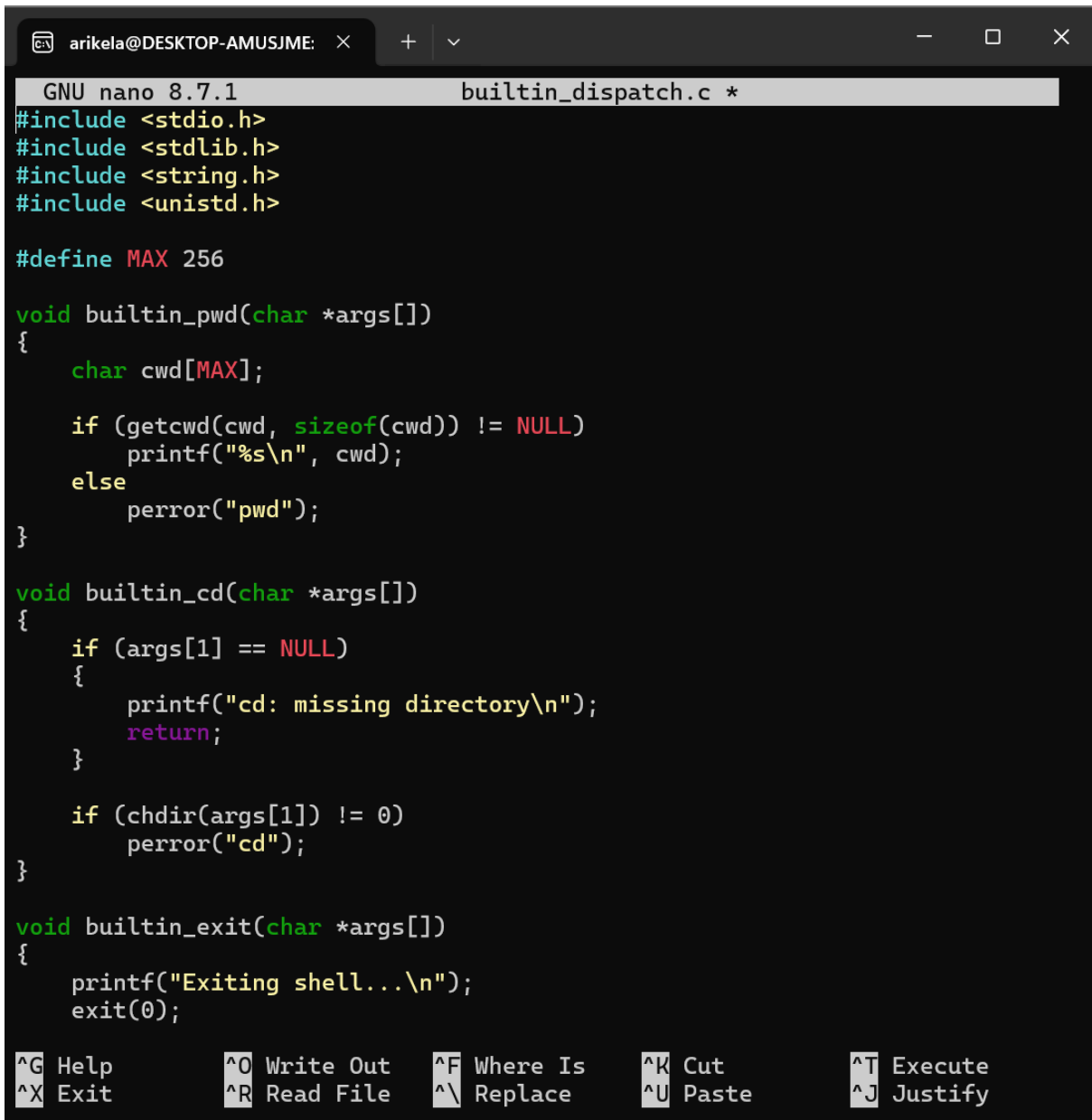
^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

arikel@DESKTOP-AMUSJME:~/skillweek7$ nano variable_expand.c
arikel@DESKTOP-AMUSJME:~/skillweek7$ gcc variable_expand.c -o variable_exp
and
arikel@DESKTOP-AMUSJME:~/skillweek7$ ./variable_expand
Enter text: my home is $HOME
Expanded result: my home is /home/arikel
arikel@DESKTOP-AMUSJME:~/skillweek7$ ./variable_expand
Enter text: current user is $USER
Expanded result: current user is arikel
arikel@DESKTOP-AMUSJME:~/skillweek7$ ./variable_expand
Enter text: Value is $DOES_NOT_EXIST
Undefined variable: DOES_NOT_EXIST
Expanded result: Value is
arikel@DESKTOP-AMUSJME:~/skillweek7$ ./variable_expand
Enter text: Home directory: ${HOME}
Expanded result: Home directory: /home/arikel
arikel@DESKTOP-AMUSJME:~/skillweek7$

```

Task-2: To Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic



```
GNU nano 8.7.1 builtin_dispatch.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX 256

void builtin_pwd(char *args[])
{
    char cwd[MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_cd(char *args[])
{
    if (args[1] == NULL)
    {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(args[1]) != 0)
        perror("cd");
}

void builtin_exit(char *args[])
{
    printf("Exiting shell...\n");
    exit(0);
}
```

 ^G Help	 ^O Write Out	 ^F Where Is	 ^K Cut	 ^T Execute
 ^X Exit	 ^R Read File	 ^\ Replace	 ^U Paste	 ^J Justify

```

void builtin_exit(char *args[])
{
    printf("Exiting shell...\n");
    exit(0);
}

void execute_builtin(char *args[])
{
    struct Builtin
    {
        const char *name;
        void (*function)(char *[]);
    };

    struct Builtin builtins[] =
    {
        {"pwd", builtin_pwd},
        {"cd", builtin_cd},
        {"exit", builtin_exit}
    };

    int count = sizeof(builtins) / sizeof(builtins[0]);

    for (int i = 0; i < count; i++)
    {
        if (strcmp(args[0], builtins[i].name) == 0)
        {
            builtins[i].function(args);
            return;
        }
    }

    printf("Invalid command: %s\n", args[0]);
}

```

```

        printf("Invalid command: %s\n", args[0]);
    }
int main()
{
    char input[MAX];

    printf("Simple Built-in Shell\n");

    while (1)
    {
        char *args[10];
        int count = 0;

        printf("shell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        char *token = strtok(input, " ");

        while (token != NULL && count < 9)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        if (count > 0)
            execute_builtin(args);
    }
}

```

```

        while (token != NULL && count < 9)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        if (count > 0)
            execute_builtin(args);
    }

    return 0;
}

```

^{^G} Help
^{^X} Exit

^{^O} Write Out
^{^R} Read File

^{^F} Where Is
^{^_} Replace

^{^K} Cut
^{^U} Paste

^{^T} Execute
^{^J} Justify

```
arikel@DESKTOP-AMUSJME:~/skillweek7$ nano builtin_dispatch.c
arikel@DESKTOP-AMUSJME:~/skillweek7$ gcc builtin_dispatch.c -o builtin_dispatch
arikel@DESKTOP-AMUSJME:~/skillweek7$ ./builtin_dispatch
Simple Built-in Shell
shell> pwd
/home/arikel/skillweek7
shell> cd/tmp
Invalid command: cd/tmp
shell> pwd
/home/arikel/skillweek7
shell> cd /tmp
shell> pwd
/tmp
shell> cd /does/not/exist
cd: No such file or directory
shell> hello
Invalid command: hello
shell> exit
Exiting shell...
arikel@DESKTOP-AMUSJME:~/skillweek7$
```